

How To Train Your Large Language Model

Final Presentation

Yven de Buhr, Jamil Mounzer, Joel Dag,
Sashreek Nayak Dhaimodkar, Luke Friedrichs, Martin Schröder

Tutors: Nikit Srivastava, Rene Speck

Supervisor: Prof. Dr. Axel Ngonga



Data Science Group
Paderborn University

February 5, 2026

Introduction

Large Language Models

Rapid Knowledge Access



Productivity & Automation

Flexibility

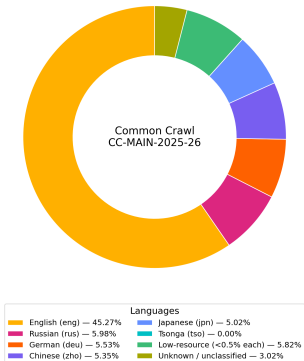


Multilingual Communication

Challenges

Language Imbalance

Common Crawl language distribution^a

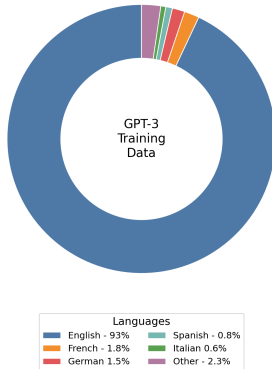


^aCommon Crawl Foundation et al. (2025)

Challenges

Language Imbalance

GPT-3 training data distribution^a

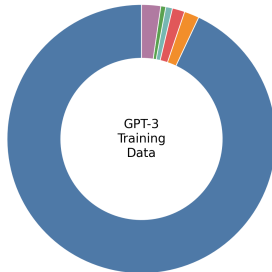


^a Johnson et al. (2022)

Challenges

Language Imbalance

GPT-3 training data distribution^a



Languages	
English - 93%	Spanish - 0.8%
French - 1.8%	Italian 0.6%
German 1.5%	Other - 2.3%

^a Johnson et al. (2022)

English-centric bias^a

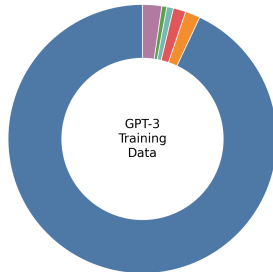
- ▶ Anglophone worldview dominates.
- ▶ Other cultural norms underrepresented.

^a Bender et al. (2021)

Challenges

Language Imbalance

GPT-3 training data distribution^a



Languages	
English - 93%	Spanish - 0.8%
French - 1.8%	Italian 0.6%
German 1.5%	Other - 2.3%

^a Johnson et al. (2022)

Performance disparity^a

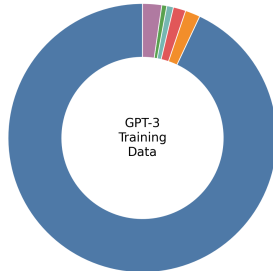
- Lower accuracy on low-resource languages.

^a Blasi et al. (2022)

Challenges

Language Imbalance

GPT-3 training data distribution^a



Languages	
English - 93%	Spanish - 0.8%
French - 1.8%	Italian 0.6%
German 1.5%	Other - 2.3%

^a Johnson et al. (2022)

Technological exclusion^a

- **88.38%** of tracked languages have zero data.

^a Joshi et al. (2020)

Challenges

Curse of multilinguality

"As dataset sizes increase, adding multilingual data begins to **hurt performance** for **both** low-resource and high-resource languages (the "*curse of multilinguality*")".¹

¹Chang et al. (2023)

Task: Train a large and **open-source multilingual** language model and address the challenges posed by the *curse of multilinguality*.

- ▶ Support 500+ languages
- ▶ Ensure computational efficiency
- ▶ Enable multimodal capabilities
- ▶ Maintain linguistic extensibility

Workflow & Tools

1. Element Messaging Platform
2. Kanban Board
3. Sciebo Cloud Storage

Infrastructure

Permanent access

- ▶ **Noctua 2** Cluster (128× NVIDIA A100 40 GB)
- ▶ **LOLA** GPU VM (2× NVIDIA H100-80GB)

Limited access

- ▶ **enexa2** GPU VM (2× *NVIDIA A100 80GB*)
- ▶ **morel** GPU VM (4× *NVIDIA H100 96GB*)
- ▶ **otus** Cluster (108× *NVIDIA H100 94GB*)

Pretraining Approach

Section 1 - Introduction to DynMoE

Train an MoE language model from scratch that is **compute-efficient** and **stable**

- ▶ **Why MoE?**

Activate only a subset of experts per token → more capacity for similar compute

- ▶ **Our motivation**

Go beyond fixed Top- k routing and actively shape routing behavior

*This led us to explore **Dynamic MoE (DynMoE)***

Key Idea

Routing is not fixed.

It adapts during training to improve **stability** and **expert utilization**.

Intuition

- ▶ Easy tokens → fewer experts
- ▶ Hard tokens → more experts
- ▶ Learn *when* to spend compute

Why it matters

- ▶ Less manual Top- k tuning
- ▶ Healthier expert usage
- ▶ Reduced early collapse

Section 2 - Gating Architecture

Dynamic-K Routing

Dynamic-K: Forward Pass

Problem: Standard Top-K forces fixed k experts.

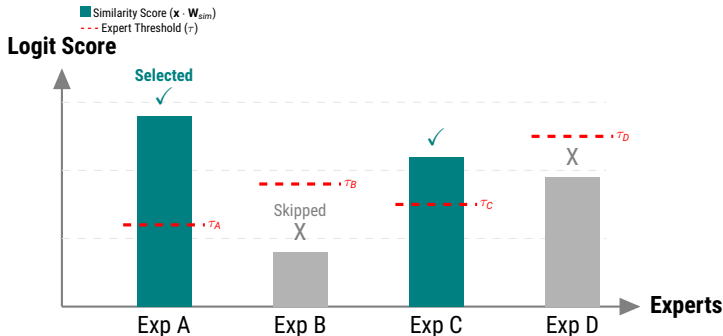
Solution: Threshold-based gating allows $1 \dots E$ experts.

Per-Expert Gating Logic:

$$\text{Keep Expert}_i \iff \underbrace{\sigma(\mathbf{x} \cdot \mathbf{w}_{sim,i})}_{\text{Token Similarity}} > \underbrace{\sigma(w_{gate,i})}_{\text{Expert Threshold}}$$

Status:

- **Active (1):** Routed
- **Dropped (0):** Skipped



The Optimization Challenge:

Binary decisions have zero gradients².

Gradient Estimation (STE):

We use the **Straight-Through Estimator**.

- ▶ **Forward:** Use the hard binary mask (0 or 1).
- ▶ **Backward:** Pretend we used a smooth Sigmoid.

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} \approx \frac{\partial \mathcal{L}}{\partial \text{hard}} \cdot \underbrace{\sigma'(\text{soft})}_{\text{Sigmoid Derivative}}$$

²Undefined at the threshold (discontinuity).

Total Training Objective:

$$\mathcal{L}_{\text{Total}} = \mathcal{L}_{\text{CE}} + \alpha \cdot \underbrace{\left(E \sum_{i=1}^E f_i^2 \right)}_{\mathcal{L}_{\text{Aux}}}, \quad f_i = \frac{1}{N} \sum_{t=1}^N \mathbf{1}\{\text{token } t \text{ is routed to expert } i\}$$

Why $\mathcal{L}_{\text{Aux}}$ forces Sparsity ($k \downarrow$):

Assuming balanced experts, $f_i \approx \frac{k}{E}$. The loss becomes:

$$\sum_{i=1}^E f_i^2 \approx \sum_{i=1}^E \left(\frac{k}{E} \right)^2 = \frac{k^2}{E} \Rightarrow \mathcal{L}_{\text{Aux}} \approx k^2$$

Result

Doubling density (k) **quadruples** the penalty.

Section 3 - Implementation Details

DynMoe vs. Ours

Scenario: 3 Tokens (T), 4 Experts (E).

Input Logits (Binary Gates):

$$\begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{array}{l} \leftarrow \text{Token 0 (picks E0, E1)} \\ \leftarrow \text{Token 1 (picks E1, E2, E3)} \\ \leftarrow \text{Token 2 (picks E0)} \end{array}$$

Expert Counts:

$$[2, 2, 1, 1] \rightarrow \text{Max Capacity}(C) = 2$$

1. Cumsum & Masking:

$$\text{loc1} = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 1 & 2 & 1 & 1 \\ 2 & 2 & 1 & 1 \end{bmatrix} \rightarrow \text{loc1}_s = \begin{bmatrix} 0 & 0 & -1 & -1 \\ -1 & 1 & 0 & 0 \\ 1 & -1 & -1 & -1 \end{bmatrix}$$

2. One-Hot Expansion (The Bottleneck): Tensor of shape $(T, E, C + 1)$.

Example Slice for **Token 0**:

$$\text{T0_Slice} = \begin{bmatrix} [1, 0, 0] & (\text{E0, Slot 0}) \\ [1, 0, 0] & (\text{E1, Slot 0}) \\ [0, 0, 1] & (\text{E2, Not picked}) \\ [0, 0, 1] & (\text{E3, Not picked}) \end{bmatrix}$$

Memory Complexity: $O(T \times E \times C)$

$C \approx T(\text{full capacity}) \implies O(T^2 E)$.

1. Find Active Indices

Select non-zero entries (marked in **red**).

$$\begin{bmatrix} \mathbf{1} & \mathbf{1} & 0 & 0 \\ 0 & \mathbf{1} & \mathbf{1} & \mathbf{1} \\ \mathbf{1} & 0 & 0 & 0 \end{bmatrix}$$

→ Total Active (N_{active}) = **6**

2. Store as Vectors

Vectors of length N_{active} :

Idx	token_idx	expert_idx	slot_idx
0	0	0	0
1	0	1	0
2	1	1	1
3	1	2	0
4	1	3	0
5	2	0	1

Memory Complexity: $O(N_{active})$

Worst Case ($N_{active} = T \times E$): i.e. every token picks every expert.

► **Memory Complexity:**

- **DynMoe:** $O(T \times E \times C)$.
- **Ours:** $O(N_{active})$.

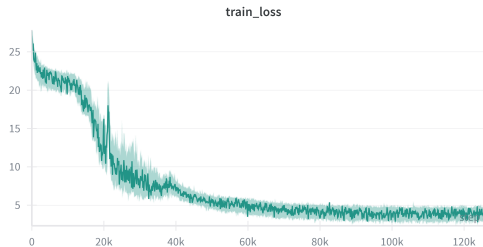
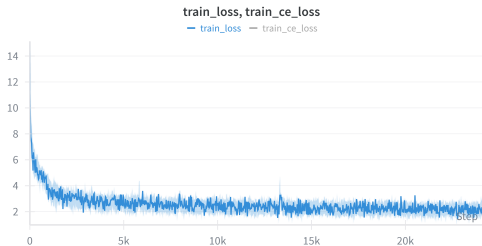
► **The "Full Capacity" Risk (Worst Case):**

- Having a maximum capacity to cover all tokens ($C \approx T$):
- **DynMoe** $\rightarrow O(T \times E \times T) = O(T^2E)$ (**Quadratic Explosion**).
- **Ours** $\rightarrow O(T \times E)$ (**Remains Linear**).

Training Run: Dense vs DynMoE

GPT-2 Dense

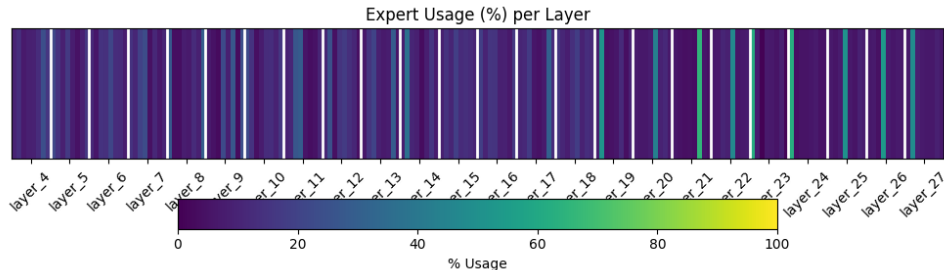
DynMoE (Dynamic- k)



Model Configs (Summary)

- **Dense:** 30L, 3.81B params, $d=3072$, heads=24, $lr=1e^{-4}$
- **DynMoE:** 28L, 3.58B params, 8 experts, dynamic- k , STE gating
- Same data (400B tokens), 128 languages, seq=2048, epochs=1,

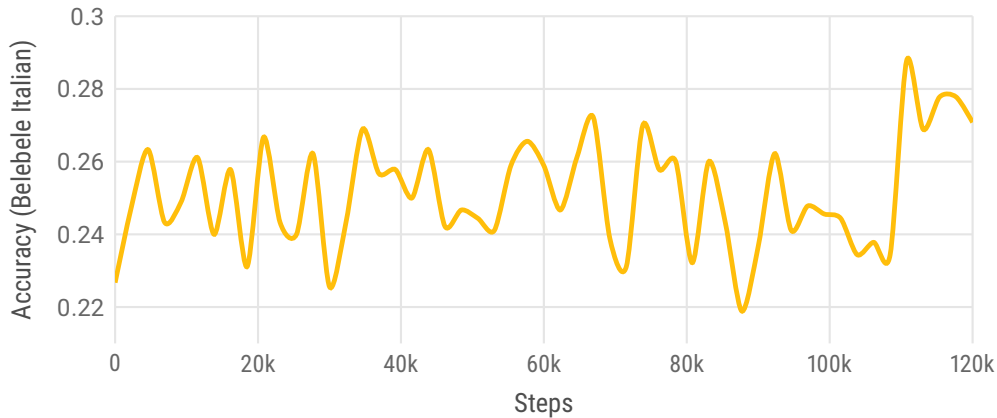
Expert Utilization Across Layers



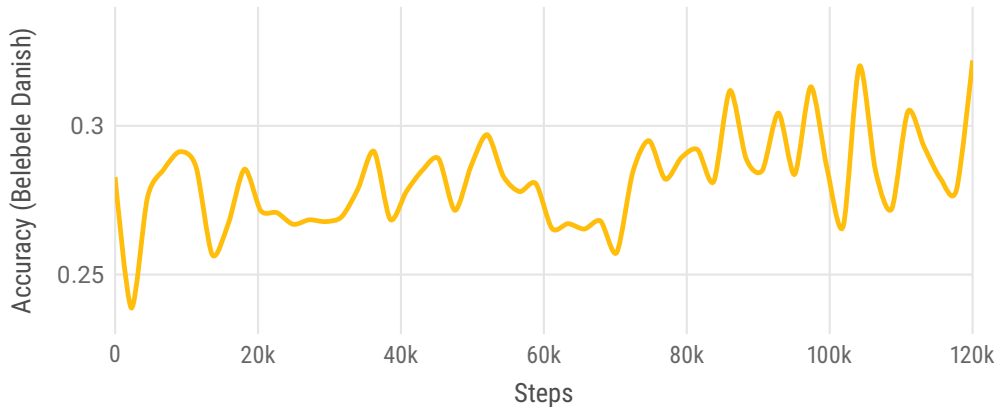
Observation

- ▶ All experts are active across depth
- ▶ No single-expert domination
- ▶ No permanently inactive experts
- ▶ sparse with average top-k ≈ 1.5

Evaluation Snapshot



Evaluation Snapshot



Section 2 - Adapter Finetuning Approach

Adapter Finetuning

Adapter Finetuning Overview

Goal, motivation, approach

Goal: Parameter-efficient multilingual adaptation of LLMs, without catastrophic interference.

Goal, motivation, approach

Goal: Parameter-efficient multilingual adaptation of LLMs, without catastrophic interference.

Motivation

- ▶ Full fine-tuning is too costly.
- ▶ Adding many languages can degrade existing languages.
- ▶ Using shared weights across all languages causes cross-language interference.

Adapter Finetuning Overview

Goal, motivation, approach

Goal: Parameter-efficient multilingual adaptation of LLMs, without catastrophic interference.

Motivation

- ▶ Full fine-tuning is too costly.
- ▶ Adding many languages can degrade existing languages.
- ▶ Using shared weights across all languages causes cross-language interference.

Approach (broadly)

- ▶ Hierarchical adapter experts: route to family expert, then subexpert.
- ▶ Low-rank adapters (LoRA): train only $\sim 0.3-4\%$ of weights.
- ▶ Language-prior routing: guide samples toward the correct expert (via language IDs).
- ▶ Build on top of SOTA adapter approaches.

Full fine-tuning (per layer)

- Update W 🔥: $W \leftarrow W + \Delta W$



Full fine-tuning
 ΔW

Adapters Foundations

Full fine-tuning (per layer)

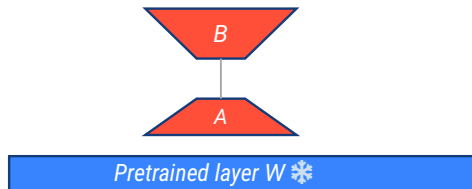
- Update W 🔥: $W \leftarrow W + \Delta W$



(a) Full fine-tuning
 ΔW

LoRA (PEFT, per layer)

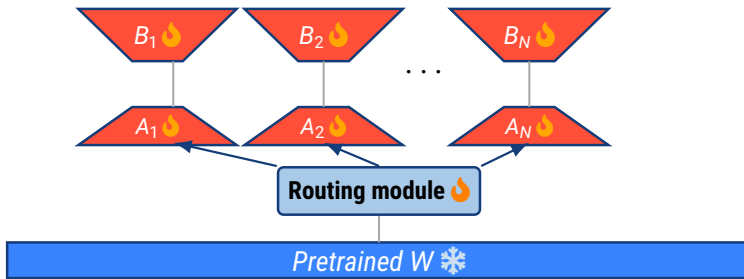
- Freeze W ❄️; learn $\Delta W = BA$ (rank r)



(b) LoRA
 $\Delta W = BA$

Adapter Architectures

Mixture of Adapters

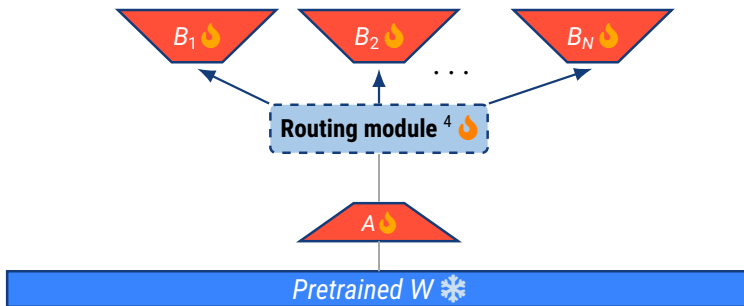


(c) **LoRA + MoE**³
 $(\Delta W = B_1 A_1 + \dots + B_N A_N)$

³Liu et al. (2024) [1]

Adapter Architectures

Asymmetric LoRA (HydraLoRA / CoLA)



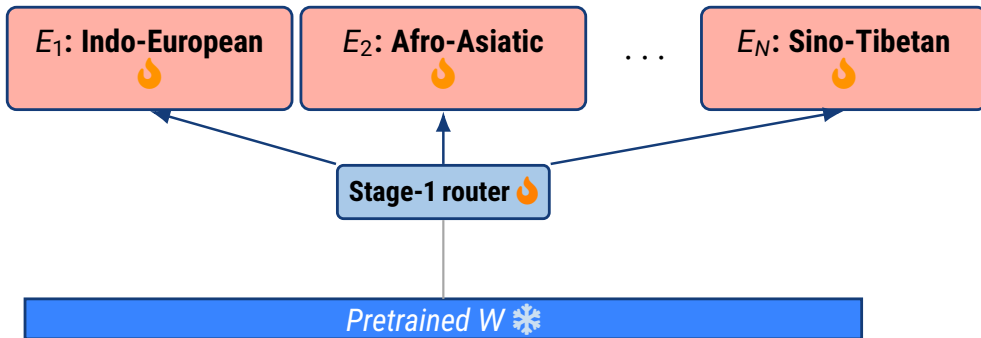
(d) HydraLoRA / CoLA⁵

$$(\Delta W = (B_1 + \dots + B_N)A)$$

⁴ Routing is optional. If not active, we sum up the B matrices

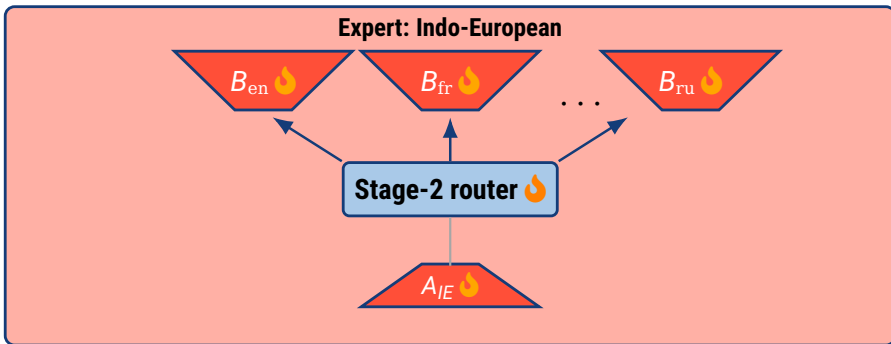
⁵ Tian et al. (2024) [2]; Zhou et al. (2025) [3].

Custom-designed training architecture - 1



⁵ $E_1/E_2/E_N = \text{Expert}_1/2/N$

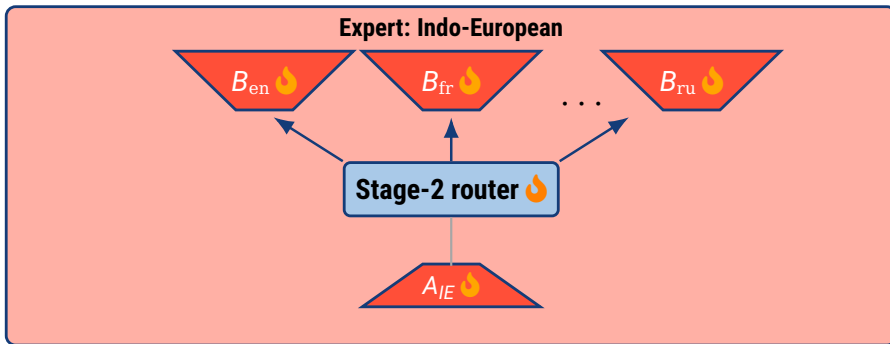
Custom-designed training architecture - 2



⁶ Blaschke et al. (2025) [4]

⁷ Tian et al. (2024) [2] en = English; fr = French; ru = Russian; IE = Indo-European

Custom-designed training architecture - 2



- ▶ Similar languages in one expert enable cross-lingual transfer learning.⁶
- ▶ Shared A matrices capture common patterns across languages.
- ▶ Separate B learns language-specific patterns.⁷

⁶ Blaschke et al. (2025) [4]

⁷ Tian et al. (2024) [2] en = English; fr = French; ru = Russian; IE = Indo-European

Language-Prior Routing Loss

Overview + our expert-router formulation

1) Motivation

Goal: keep routing language-aligned.

- ▶ With many experts/heads, routers can **drift** and misroute languages $\Rightarrow$ interference.
- ▶ LPR adds a lightweight **language-supervised** signal that nudges routing toward the intended expert.

⁷ Language-prior routing loss adapted from MoE-LPR Zhou et al. (2024) [5].

Language-Prior Routing Loss

Overview + our expert-router formulation

1) Motivation 2) Original (MoE-LPR)

Original use (MoE-LPR, review stage).

- ▶ Dense model → MoE: base expert is frozen; new experts are trained on expanded languages.
- ▶ Review stage: **train only the router** with a small replay set to reduce forgetting.
- ▶ Prior: original-language tokens should route to the **frozen base expert** (expert 0).

$$L_{\text{LPR}}(W_r) = - \sum_{t \in \mathcal{B}} \mathbf{F}(t) \log G_0(t).$$

$\mathbf{F}(t)=1$ iff token t is from an original language.

⁷ Language-prior routing loss adapted from MoE-LPR Zhou et al. (2024) [5].

Language-Prior Routing Loss

Overview + our expert-router formulation

1) Motivation 2) Original (MoE-LPR) 3) Our loss (expert)

Our adaptation (step 1/3): define targets.

- ▶ Batch $\mathcal{B}$ contains examples indexed by i ; each has one language ID ℓ_i .
- ▶ Map language to target expert family: $g(\ell_i) = \mathbf{e}_i$.

Intuition: “this language should use that expert family”.

⁷ Language-prior routing loss adapted from MoE-LPR Zhou et al. (2024) [5].

Language-Prior Routing Loss

Overview + our expert-router formulation

1) Motivation 2) Original (MoE-LPR) 3) Our loss (expert)

Our adaptation (step 2/3): aggregate router evidence.

- ▶ Token-level expert-router logits: $z_{i,t} \in \mathbb{R}^E$ (t = token index; T_i tokens in example i).
- ▶ Aggregate to one example-level score:

$$\bar{z}_i = \frac{1}{T_i} \sum_{t=1}^{T_i} z_{i,t}.$$

⁷ Language-prior routing loss adapted from MoE-LPR Zhou et al. (2024) [5].

Language-Prior Routing Loss

Overview + our expert-router formulation

1) Motivation 2) Original (MoE-LPR) 3) Our loss (expert)

Our adaptation (step 3/3): expert-router LPR loss.

$$L_{\text{LPR}}^{\text{expert}} = - \sum_{i \in \mathcal{B}} \mathbf{1}[e_i \geq 0] \log \text{softmax}(\bar{z}_i)_{e_i}.$$

- ▶ $\mathbf{1}[e_i \geq 0]$: skip unknown targets.
- ▶ Training objective: $L = L_{\text{NTP}} + \lambda L_{\text{LPR}}^{\text{expert}}$.
- ▶ “penalize the router when it doesn’t select the right expert” (softly, via cross-entropy).
- ▶ LPR is **soft** supervision; routing is guided, not forced.

⁷ Language-prior routing loss adapted from MoE-LPR Zhou et al. (2024) [5].

Language-Prior Routing Loss

Overview + our expert-router formulation

1) Motivation 2) Original (MoE-LPR) 3) Our loss (expert) 4) Tradeoffs

► Pros:

- Similar languages share experts → cross lingual effects, less noise.
- Enables high modularity and extensibility: Add new experts/subexperts and steer traffic to them via routing (and LPR), reducing interference with existing experts.

► Cons:

- Requires language IDs. Can lead to unbalanced routing.
- Can over-bias routing: Language routing may contradict models language understanding.

⁷ Language-prior routing loss adapted from MoE-LPR Zhou et al. (2024) [5].

Experimental Setup

Training conditions

Dataset

- ▶ Token budget: 12B tokens
- ▶ Base model: Llama 3.1-8B

Baselines & Tiers

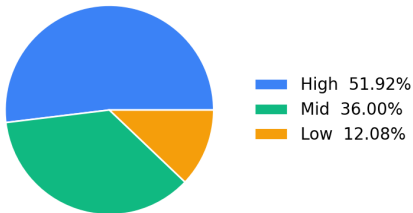
- ▶ Baselines: CoLA, HydraLoRA, LoRA
- ▶ Routing tiers: hard / LPR / expert-only
- ▶ Experts: 11 total
- ▶ B heads: 2-11; top- $k=1$

Eval

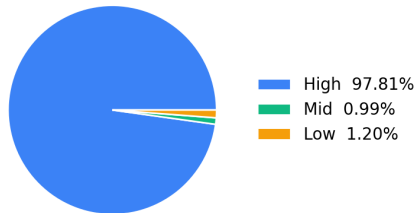
- ▶ Belebele (122 langs)
- ▶ Flores perplexity (200 langs)
- ▶ Eval Loss - 2% of training data

⁷ Experts and the language mapping were derived using language clustering based on URIEL distances (Littel et al., 2017).

- ▶ 12B tokens; 200 languages sampled from FineWeb2⁸
- ▶ Resource tiers: 25H / 16M / 159L.⁹; Applied language distribution smoothing¹⁰



F1 a): Training resource distribution



F1 b): Original FineWeb resource distribution

⁸ Penedo et al., 2025: FineWeb2

⁹ H/M/L = high/mid/low-resource languages

¹⁰ Xue et al. (2021): mT5

Main Results 10% training data

Evaluation Loss across approaches

Adapter	Variant	Trainable	Eval loss
LoRA	flat	13.6M (0.17%)	5.3929
CoLA	head-bias	323.5M (3.87%)	5.3786
CoLA	hard	323.5M (3.87%)	6.5719
CoLA	LPR	323.5M (3.87%)	8.5542
HydraLoRA	flat	25.7M (0.32%)	5.3903
HydraLoRA	expert LPR	346.0M (4.13%)	5.3775
HydraLoRA	expert LPR (expert-only)	346.0M (4.13%)	5.3782
HydraLoRA	hard	346.0M (4.13%)	5.7634
CoLA	baseline (flat)	24.1M (0.30%)	5.4023

Benchmark results 10% training data

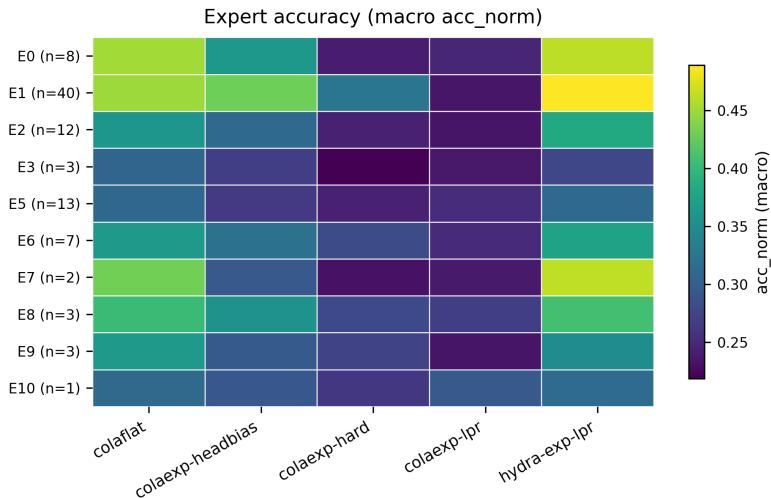
Flores perplexity & Belebele accuracy

approach	Belebele acc_norm (all)	Belebele high	Belebele mid	Belebele low	FLORES byte_ppl (all)	FLORES high	FLORES mid	FLORES low
Cola Experts LPR Headbias	0.3523	0.4594	0.3810	0.2942	2.5115	2.2425	2.1127	3.2564
Cola Experts LPR	0.2339	0.2328	0.2355	0.2340	10.5121	7.4636	11.6371	18.0137
hydra_hydra-exp-hard	0.4022	0.5294	0.4750	0.3230	3.0016	2.3493	2.0962	4.7979
HydraLoRa Experts Only LPR	0.4245	0.5706	0.4555	0.3474	2.5798	2.2664	2.1439	3.4428
HydraLoRa Experts Only LPR Two Stage	0.4224	0.5721	0.4577	0.3424	2.5765	2.2646	2.1465	3.4350
CoLA Flat / Baseline	0.3917	0.5125	0.4072	0.3306	2.6291	2.2761	2.1503	3.5993

Belebele acc_norm macro across approaches; **bold** = best

¹⁰ belebele acc_norm: normalized acc by averaging log-likelihood per answer token, to remove answer-length and cross-language tokenization bias, and they recommend it as the official metric.

Expert-Level Analysis



¹⁰

Metric used is belebele acc_norm over all trained languages. Languages performance are grouped and averaged according to derived language expert clusters

Best vs Worst Experts (HydraLoRA LPR vs CoLA)

Largest Expert performance changes

Largest gains (↑)	Largest drops (↓)
E1 +0.0578 (Indo-European) <i>English, German, Spanish, French, Russian</i>	E3 -0.0411 (Sino-Tibetan) <i>Burmese, Tibetan, Kachin</i>
E7 +0.0450 (Uralic) <i>Hungarian, Finnish</i>	E9 -0.0033 (Kra-Dai) <i>Thai, Lao, Shan</i>
E0 +0.0200 (Mixed) <i>Japanese, Korean, Vietnamese, Norwegian Bokmål, Kannada</i>	–
Difference is Belebele acc_norm (HydraLoRA exp-LPR minus CoLA flat), 92-language intersection.	

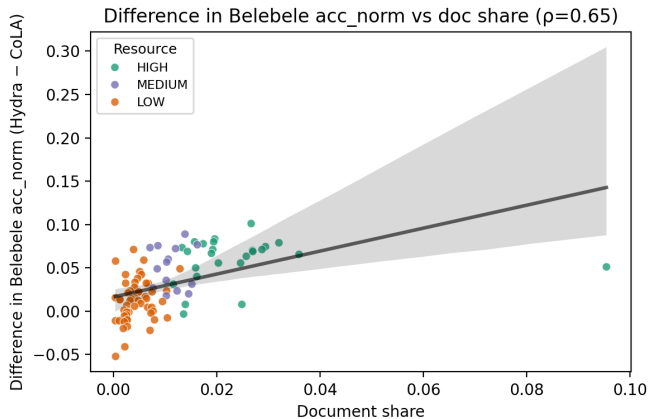
Why do gains cluster?

Data volume vs routing quality

- ▶ We analyze per-language Δ belebele acc_norm: (HydraLoRA expert-LPR — CoLA flat).
- ▶ Hypothesis: gains depend on (i) **available data** and (ii) **routing quality**.
- ▶ Unchecked hypothesis: (iii) expert mapping does (not) resembles models language understading

Data volume drives gains

Belebele: acc_norm difference vs. amount of documents

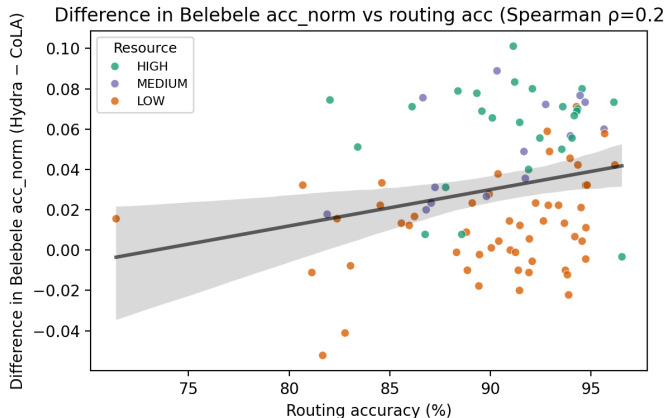


Black line: linear fit; grey band: 95% CI (Spearman $\rho = 0.65$)

- Each point = one language (92 total)
- **x**: document share **y**: $\Delta\text{acc_norm}$ (Hydra - CoLA)
- **Trend**: higher document count $\Rightarrow$ larger Hydra-LPR gains
- Our Hydra-LPR approach outperforms CoLA more as x increases

Routing quality helps (weaker)

Belebele: acc_norm difference vs. routing accuracy



- Each point = one language (92 total)
- **x**: routing accuracy **y**: $\Delta\text{acc_norm}$ (Hydra - CoLA)
- **Trend**: higher routing accuracy $\Rightarrow$ slightly larger Hydra-LPR gains
- Effect is positive but weaker than data-volume

HydraLoRA Expert-Routing Analysis

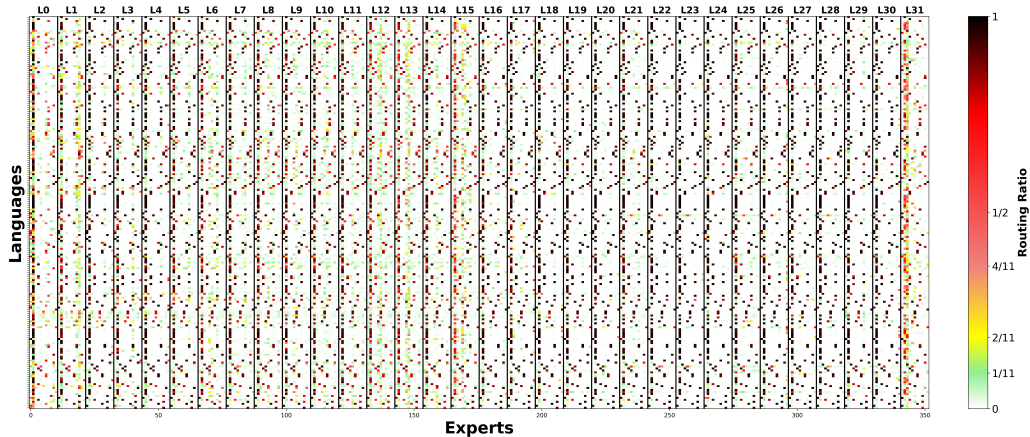


Figure: Actual Expert Routing for Hydralora-LPR; shows the learned routing.

Adapter Finetuning

Takeaways

Goal & Method

- ▶ PEFT for 200 languages with less interference.
- ▶ Hierarchical adapter experts (family → subexpert).
- ▶ Expert-router LPR: soft language guidance for routing.

What Works

- ▶ Best results come from **expert-LPR** routing.
- ▶ Gains cluster in higher-data language groups.
- ▶ Data share correlates strongly with improvements ($\rho \approx 0.65$).

Limits & Next

- ▶ Low-resource clusters can starve / regress.
- ▶ Needs language IDs + language → expert mapping.
- ▶ Mitigate with load-balancing and better priors / schedules.

Discussion :) / Questions?

Repository

<https://github.com/dice-group/HTYLLM-PG/>



Scan for code + slides.

- [1] Q. Liu, X. Wu, X. Zhao, Y. Zhu, D. Xu, F. Tian, and Y. Zheng, “When MoE meets LLMs: Parameter efficient fine-tuning for multi-task medical applications,” in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 1104–1114, 2024.
- [2] C. Tian, Z. Shi, Z. Guo, L. Li, and C. Xu, “Hydralora: An asymmetric lora architecture for efficient fine-tuning,” *arXiv preprint arXiv:2404.19245*, 2024.
- [3] Y. Zhou, C. Yao, and J. Chen, “Cola: Collaborative low-rank adaptation,” *arXiv preprint arXiv:2505.15471*, 2025.
- [4] V. Blaschke, M. Fedzechkina, and M. ter Hoeve, “Analyzing the effect of linguistic similarity on cross-lingual transfer: Tasks and experimental setups matter,” *arXiv preprint arXiv:2501.14491*, 2025.

- [5] H. Zhou, Z. Wang, S. Huang, X. Huang, X. Han, J. Feng, C. Deng, W. Luo, and J. Chen, "Moe-lpr: Multilingual extension of large language models through mixture-of-experts with language priors routing," *arXiv preprint arXiv:2408.11396*, 2024.
- [6] HuggingFaceFW, "Fineweb2 dataset."
<https://huggingface.co/datasets/HuggingFaceFW/fineweb-2>, 2024.

Appendix

Suggestion: Take this as beackup slide

LoRA

$$W = W_0 + BA \quad (1)$$

HydraLoRA (multi-head)

$$\Delta W(x) = \sum_{h=1}^H \alpha_h(x) B_h(Ax) \quad (2)$$

CoLA (collaborative)

$$\Delta W(x) = \left(\sum_{j=1}^B B_j \right) \left(\sum_{i=1}^A A_i x \right) \quad (3)$$

Tokenizer Extension Pipeline

- ▶ Base coverage: *pieces-per-word, chars-per-token, unknown* rate.
- ▶ Allocation: convert inefficiency into per-language token budget.
- ▶ Optional multilingual training for target vocab size.
- ▶ Extension: merge base + multilingual tokens; initialize new embeddings.
- ▶ Extended coverage + comparison for sanity checks.

Sentence: “I like pizza”

Experts: IE (0), ST (1), Sem (2)

LPR loss (fixed):

$$L_{\text{LPR}} = - \sum_{i \in \mathcal{B}} \mathbf{1}[e_i \geq 0] \log (\text{softmax}(\bar{\mathbf{z}}_i)_{e_i})$$

Language prior:

$$\ell = \text{English} \Rightarrow e = 0$$

Sentence: “I like pizza”

Experts: IE (0), ST (1), Sem (2)

LPR loss (fixed):

$$L_{\text{LPR}} = - \sum_{i \in \mathcal{B}} \mathbf{1}[e_i \geq 0] \log (\text{softmax}(\bar{z}_i)_{e_i})$$

Token router logits:

$$z_t \in \mathbb{R}^3 : [2.0, 0.5, 0.2], [1.8, 0.6, 0.1], [2.2, 0.4, 0.3]$$

Sentence: “I like pizza”

Experts: IE (0), ST (1), Sem (2)

LPR loss (fixed):

$$L_{\text{LPR}} = - \sum_{i \in \mathcal{B}} \mathbf{1}[e_i \geq 0] \log (\text{softmax}(\bar{z}_i)_{e_i})$$

Aggregate (sentence-level):

$$\bar{z} = \frac{1}{3} \sum_t z_t = [2.0, 0.5, 0.2]$$

Sentence: “I like pizza”

Experts: IE (0), ST (1), Sem (2)

LPR loss (fixed):

$$L_{\text{LPR}} = - \sum_{i \in \mathcal{B}} \mathbf{1}[e_i \geq 0] \log (\text{softmax}(\bar{z}_i)_{e_i})$$

Expert probabilities:

$$\text{softmax}(\bar{z}) = [0.73, 0.15, 0.12]$$

Sentence: “I like pizza”

Experts: IE (0), ST (1), Sem (2)

LPR loss (fixed):

$$L_{\text{LPR}} = - \sum_{i \in \mathcal{B}} \mathbf{1}[e_i \geq 0] \log (\text{softmax}(\bar{z}_i)_{e_i})$$

Apply loss:

$$L_{\text{LPR}} = -\log(0.73)$$

Cola Family wise Expert-Routing Analysis

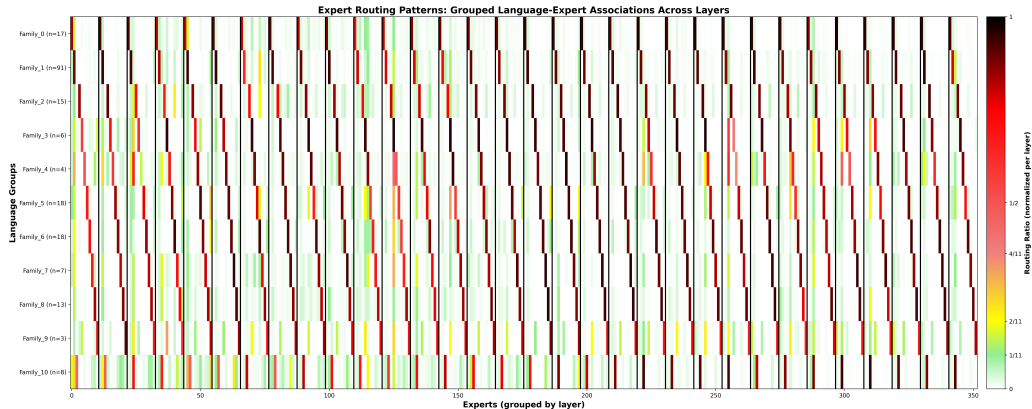


Figure: Actual Expert Routing for Cola LPR; shows the learned routing for each family.

Hydralora Family wise Expert-Routing Analysis

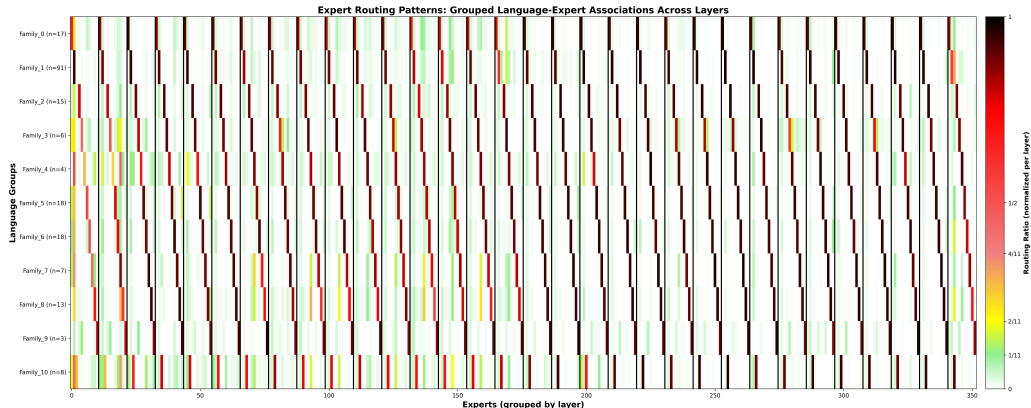
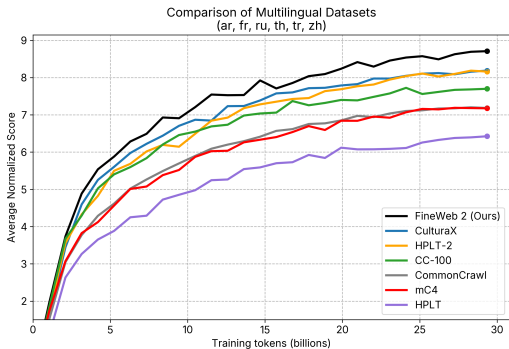


Figure: Actual Expert Routing for Hydralora LPR; shows the learned routing for each family.



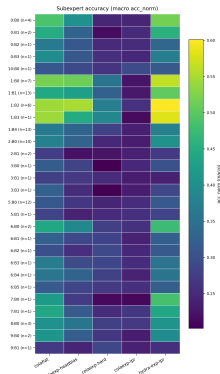
Fineweb 2 Dataset [6]

- ▶ 20TB data
- ▶ 1800 languages
- ▶ 5 billion documents

Our subsample

- ▶ 14,000,000 documents in 190 languages

Subexpert-Level Analysis



Mean acc_norm per subexpert and variant; highlights fine-grained cluster differences.

- ▶ 200-languages sampled from FineWeb2 with fixed walltime budget ¹¹
- ▶ Sampling uses temperature-style smoothing with $\alpha = 0.3$. ¹²
- ▶ Tokenization preserves language IDs for routing and analysis.
- ▶ Evaluation focuses on Belebele `acc_norm (macro)` and Flores per language.

$$w_\ell = n_\ell^{0.3}, \quad p_\ell = \frac{w_\ell}{\sum_j w_j}, \quad B_\ell = B \cdot p_\ell \quad (4)$$

¹¹Fineweb2 dataset.

¹²Conneau and Lample (2019)

Sampling Distribution

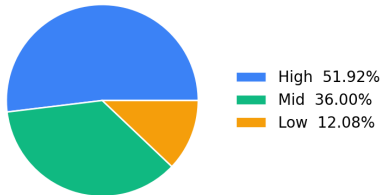
Tiered allocation

Total token budget: 12B tokens

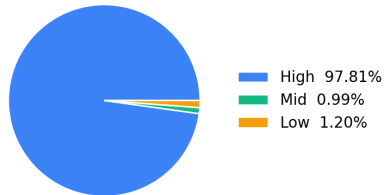
Tier	Langs	Docs (M)	Tokens (B)
High	25	10.11	6.23
Mid	16	8.23	4.32
Low	159	2.22	1.45

Table: Sample distribution after alpha smoothing and budget allocation.

Sampling Distribution Visuals



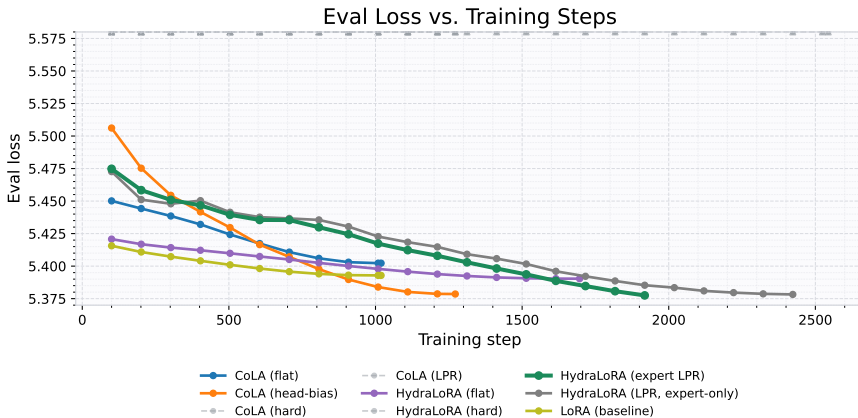
Sampling distribution



FineWeb resource distribution

Eval Loss

Comparison across adapters (10% steps)



Evaluation loss vs training steps; green = best, dashed gray = weak runs.

Best vs Worst Subexperts (HydraLoRA LPR vs CoLA)

Largest subexpert performance changes

Largest gains (↑)	Largest drops (↓)
1:B2 +0.0767 (Romance) <i>Spanish, French, Italian, Portuguese, Catalan</i>	3:B3 -0.0522 (Sino-Tibetan) <i>Kachin</i>
1:B1 +0.0711 (Slavic) <i>Russian, Polish, Ukrainian, Czech, Slovak</i>	3:B1 -0.0411 (Sino-Tibetan) <i>Tibetan</i>
1:B3 +0.0667 (Hellenic) <i>Modern Greek (1453-)</i>	0:B3 -0.0222 (Language isolate) <i>Basque</i>

Difference is Belebele acc_norm (HydraLoRA exp-LPR minus CoLA flat), 92-language intersection.

Benchmark results 100% training data

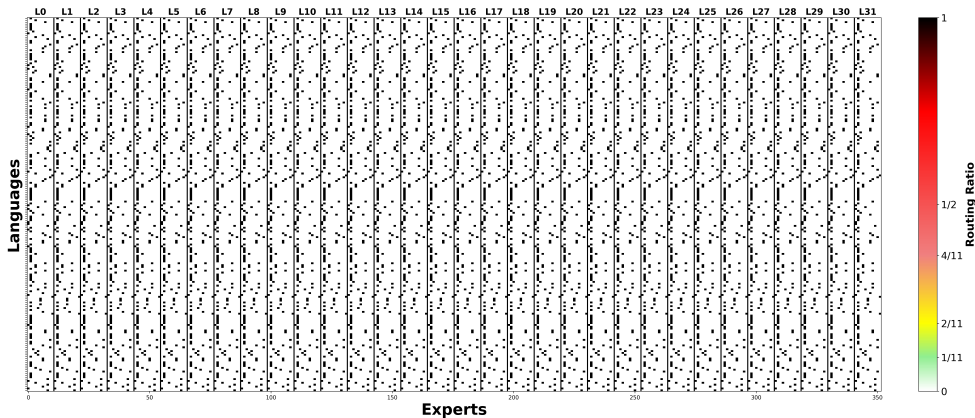
Flores perplexity & Belebele accuracy

approach	Belebele acc_norm (all)	Belebele high	Belebele mid	Belebele low	FLORES byte_ppl (all)	FLORES high	FLORES mid	FLORES low
cola_colexp-headbias (only trained 60%)	0.3573	0.4652	0.3875	0.2985	2.5152	2.2637	2.1458	3.2111
cola_colexp-lpr (only trained 60%)	0.3662	0.4619	0.3994	0.3124	2.5768	2.2779	2.1539	3.4012
hydra_hydra-exp-lpr (only trained 14%)	0.4127	0.5520	0.4488	0.3375	2.6025	2.2801	2.1618	3.4891
cola_colafat (trained 100%)	0.4134	0.5390	0.4375	0.3478	2.6332	2.2961	2.1597	3.5623

Belebele acc_norm macro; **bold** = current best (pending updates).

CoLA/HydraLoRA Expert-Routing Analysis

Target (ideal) routing



Target routing for expert-LPR: which expert each language **should** route to.

COLA/HydraLoRA Expert-Routing Analysis

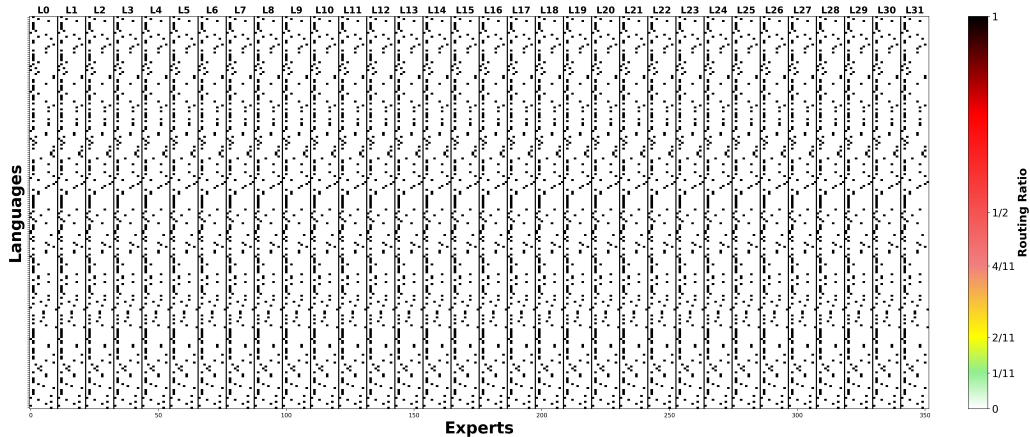


Figure: Target Expert Routing for COLA LPR and HydraLoRA LPR; shows the ideal routing.

COLA Expert-Routing Analysis

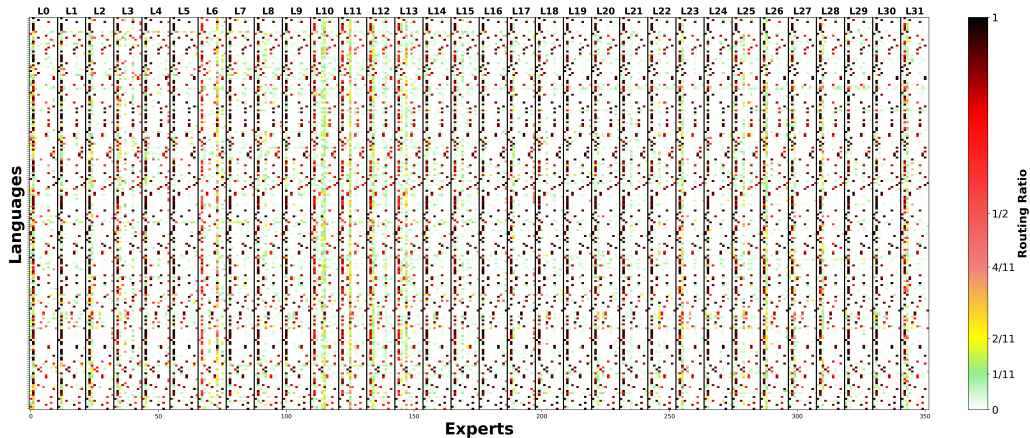
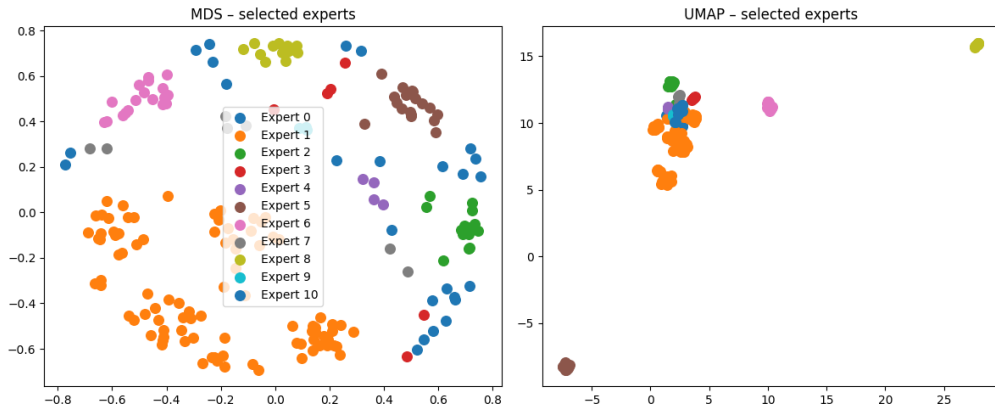
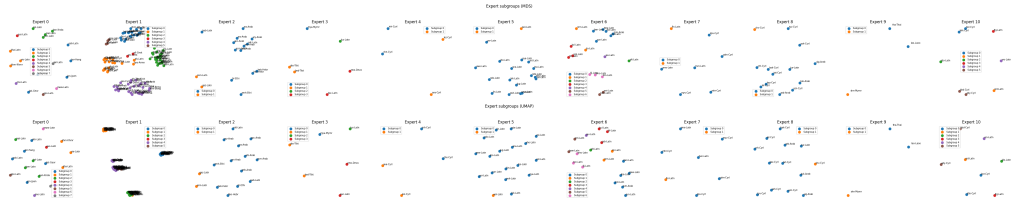
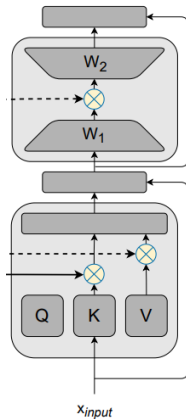


Figure: Actual Expert Routing for COLA LPR; shows the learned routing.





Transformer block



LoRA (Hu et al., 2021)

- ▶ Assume task adaptation lies in a low-rank subspace
- ▶ Reparameterize updates:

$$W_0 \in \mathbb{R}^{d \times k}, \quad W = W_0 + \Delta W, \quad \Delta W = BA, \quad B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times k}, \quad r \ll \min(d, k)$$

- ▶ Forward pass:

$$h = W_0 x + \alpha B(Ax)$$

- ▶ Train only A, B ; freeze W_0

HydraLoRA LPR - Expert Target hit rate

